Apellidos: ..... Nombre: ..... Grup: .....

Examen final IC (*Parte 2)*

- Duración del examen: 2:30 horas.
- No podéis utilizar calculadora, móvil, apuntes, etc.
- La solución del examen se publicará en Atenea mañana por la mañana.

Nota: La indicación de la puntuación de los ejercicios es sobre 10 puntos, pero esta parte del examen final solo representa 5 puntos de la nota del examen final.

Ejercicio 1 *(1.6 puntos)*

El programa en ensamblador SISA que se muestra a continuación se ha traducido a lenguaje máquina situando la sección de datos (.data) a partir de la dirección `0x0800` y la sección de código (.text) a partir de la dirección `0xA000`.

```
        .data
                long=4
                A: .word 0x000F, 0xAB00, 0x0100,0xFFFE
                B: .byte 0X03, 0x90, 0x80, 0xFC
                C: .space 100
        .text
                MOVI  R3, long
                MOVI  R1, lo(A)
                MOVHI R1, hi(A)
                MOVI  R2, lo(B)
                MOVHI R2, hi(B)
                MOVI  R7, 0
                MOVHI R7, 0x80
        loop:   LD    R4, 0(R1)
                LDB   R5, 0(R2)
                SUB   R6, R4, R5
                AND   R0, R6, R7
                BNZ   R0, neg
        pos:    ST    20(R1), R6
                BZ    R0, cont
        neg:    ST    20(R1), R5
        cont:   ADDI  R1, R1, 2
                ADDI  R2, R2, 1
                ADDI  R3, R3, -1
                BNZ   R3, loop
        .end
```

Una vez ensamblado y cargado el programa en la memoria y antes de comenzar su ejecución:

a) ¿A qué direcciones de memoria corresponden las etiquetas o direcciones simbólicas siguientes? *(0.2 puntos)*

`C=0x` `loop=0x`

b) ¿Cuál es la dirección de memoria y su contenido donde han quedado almacenadas las siguientes instrucciones una vez cargado el programa? *(0.6 puntos)*

Instrucción	@memoria y contenido
`LDB R5, 0(R2)`	Mem_w[0x]=0x
`AND R0, R6, R7`	Mem_w[0x]=0x
`BNZ R3, loop`	Mem_w[0x]=0x

c) Una vez ejecutado el programa en el SISC von Neumann, completa la siguiente tabla con el contenido de la memoria en hexadecimal (no es necesario poner **0x** del valor). Las posiciones de memoria que no se pueda saber su contenido dejadlas en blanco. *(0.4 puntos)*

@memoria	Valor
0x0800	
0x0801	
0x0802	
0x0803	
0x0804	
0x0805	
0x0806	
0x0807	

@memoria	Valor
0x0808	
0x0809	
0x080A	
0x080B	
0x080C	
0x080D	
0x080E	
0x080F	

@memoria	Valor
0x0810	
0x0811	
0x0812	
0x0813	
0x0814	
0x0815	
0x0816	
0x0817	

@memoria	Valor
0x0818	
0x0819	
0x081A	
0x081B	
0x081C	
0x081D	
0x081E	
0x081F	

d) Si se ejecuta el código mostrado, ¿Cuántas instrucciones se ejecutan, cuántas son lentas y cuántas son rápidas en la Harvard multiciclo? Si se ejecuta el código en las tres versiones de los computadores SISC (Harvard uniciclo, el Harvard multiciclo y el Von Neumann) ¿Cuántos ciclos tarda en ejecutarse el código en cada una se los tres computadores? ¿Cuánto tarda en ejecutarse el código en el Harvard uniciclo, en el Harvard multiciclo y en el Von Neumann suponiendo que los tiempos de ciclo son 3.000, 2000 y 1.000 u.t. respectivamente? *(0.4 puntos)*

Nº de instruc. ejecutadas =	Nº instr. lentas (H. multiciclo)=	Nº instr. rápidas (H. multiciclo)=
Nº de ciclos (H. uniciclo)=	Nº de ciclos (H. multiciclo)=	Nº de ciclos (Von Neumann)=
Tejec(Harvard uniciclo) =	Tejec(Harvard multiciclo) =	Tejec(Von Neumann) =

Ejercicio 2 *(1.5 puntos)*

Cada uno de los apartados pregunta sobre un ciclo concreto de la ejecución de varias instrucciones en el SISC Von Neumann. Escribid el contenido del registro IR, el contenido de la ROM_OUT y el valor de los bits de la **palabra de control** que genera el bloque **SISC CONTROL UNIT** durante el ciclo a que hace referencia cada apartado**.** Para cada apartado/fila se indica el nodo/estado de la UC en ese ciclo y la instrucción (en ensamblador) que está almacenada en el IR en ese ciclo. Podéis ver el grafo de estados de Moore de la UC en el anexo. Suponed que el contenido de todos los registros, Rk para k=0,...7, antes de ejecutarse cada instrucción es 0.

a) Indica el contenido del registro IR. Utilizad el valor 0 para los bits que sean x (solo para este apartado). *(0.3 puntos)*

Nodo / Estado (Mnemo Salida)	**Instrucción en IR** (en ensamblador)	**Valor del IR** (en hexadecimal)
D	`JALR R1,R2`	0x
Stb	`STB 3(R4),R5`	0x
Addi	`ADDI R6,R5,-3`	0x

b) Indica el contenido de la ROM_OUT en hexadecimal usando las conexiones en el orden que están en el anexo. Utilizad el valor 0 para los bits que sean x (solo para este apartado). *(0.6 puntos)*

Nodo / Estado (Mnemo Salida)	**Instrucción en IR** (en ensamblador)	**Contenido ROM_OUT** (en hexadecimal)
D	`JALR R1,R2`	0x
Stb	`STB 3(R4),R5`	0x
Addi	`ADDI R6,R5,-3`	0x

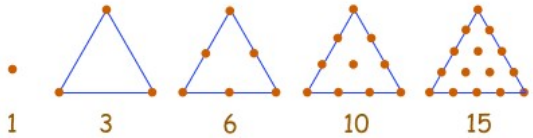
c) Indica el valor de los bits de la **palabra de control** que genera el bloque **SISC CONTROL UNIT** durante el ciclo a que hace referencia cada apartado**. Poned x siempre que no se pueda saber el valor de un bit** (ya que no podemos suponer cómo se han implementado las x en la ROM_OUT). *(0.6 puntos)*

Nodo / Estado (Mnemo Salida)	**Instrucción en IR** (en ensamblador)	**Palabra de Control**																		
		@A	@B	Pc/Rx	Ry/N	OP	F	P/I/L/A	@D	WrD	Wr-Out	Rd-In	Wr-Mem	Ldlr	LdPc	Byte	Alu/R@	R@/Pc	N (hexa)	ADDR-IO (hexa)
D	`JALR R1,R2`																			
Stb	`STB 3(R4),R5`																			
Addi	`ADDI R6,R5,-3`																			

Apellidos: ..... Nombre: ..... Grup: .....

Ejercicio 3 *(1.4 puntos)*

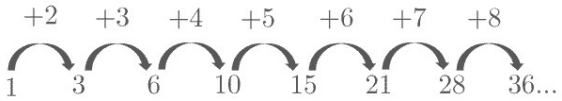
Una de las sucesiones más antiguas estudiadas es la correspondiente a los números triangulares. Un número triangular es aquel que puede recomponerse en la forma de un triángulo equilátero (por convención, el primer número triangular es el 1). Los números triangulares, junto con otros números figurados, fueron objeto de estudio por Pitágoras y los Pitagóricos, quienes consideraban sagrado el 10 escrito en forma triangular, y al que llamaban Tetraktys. Esta sucesión se genera con un patrón de puntos que forma un triángulo. Añadiendo otra fila de puntos y contando el total se encuentra el siguiente número de la sucesión.



Numéricamente la sucesión se escribe como 1, 3, 6, 10, 15, 21, 28, 36, 45, ... Donde cada número triangular T_n está definido por la siguiente fórmula: $T_n = \frac{n(n+1)}{2}$

a) Completad el código SISA para que calcule *k* primeros elementos de la sucesión de números triangulares y los almacene en memoria a partir de la posición 23456. En la posición 23456 deberá almacenarse el primer valor de la sucesión (T_0), en la 23458 el segundo valor de la sucesión (T_1) y así sucesivamente hasta *k* elementos. El código debe rellenar las *k* primeras posiciones de memoria a partir de la posición 23456 con los valores de la sucesión. Los valores son números naturales de 16 bits. En caso de que el valor de la sucesión calculado exceda los 16 bits el programa se queda con los 16 bits de menor peso. El número de valores de la sucesión a calcular (*k*) nos vendrá indicado mediante un dispositivo de teclado con efecto lateral en la lectura que tiene el registro de status en la posición 9 y el de datos en la 4. Y se nos garantiza que este valor de *k* siempre estará comprendido entre 1 y 5000. *(1 punto)*

La regla de formación de los números triangulares es que cada término se obtiene sumando al anterior la cantidad correspondiente a su número de orden, como podemos observar en la siguiente figura.




```
.data
.text     MO___   R5, _________   ; R5=primera posición de memoria
          MO___   R5, _________
          MO___   R6, _________   ; R6=primer valor
          MO___   R4, _________   ; R4=número de orden
espera:   _____   R7, _________   ; hay dato?
          _____   R7, _________
          _____   R1, _________   ; numero elementos
bucle:    _____   0(____), ____
          _____   R5,____, ____
          _____   R4,____, ____
          _____   R6,____, ____
          _____   R1,____, ____
          _____   R1, _______
.end
```

b) ¿Cuántos ciclos y unidades de tiempo tarda en ejecutarse el código anterior en Harvard uniciclo y en el Von Neumann, considerando que el número de elementos recibidos es β, que el valor β ya está disponible en los puertos de entrada/salida en el momento de empezar la ejecución del programa y que el tiempo de ciclo del Harvard uniciclo es de 4000 u.t. y el del Von Neumann es de 1000 u.t.? *(0.4 puntos)*

Código en el Harvard uniciclo: Número de ciclos= Tejec=

Código en el Von Neumann: Número de ciclos= Tejec=

Ejercicio 4 *(1.2 puntos)*

Especificad el **camino crítico** (indicando la suma ordenada de los tiempos de propagación de los bloques por los que pasa) y calculad el **tiempo de ciclo mínimo** para que el computador **SISC Von Neumann** pueda ejecutar correctamente el tipo de instrucción SISA que se indica en cada apartado (este sería el tiempo de ciclo mínimo del computador si solo ejecutara instrucciones como la indicada u otras que requieran menor tiempo). No tenéis que añadir ningún porcentaje de seguridad en el cálculo del tiempo de ciclo mínimo. Suponed que los tiempos de propagación de los bloques que forman el computador son los siguientes:

T_p(ROM_Q+) = 60 u.t.

T_p(ROM_OUT) = 80 u.t.

T_p(MUX-2-1) = 50 u.t.

T_p(MUX-4-1) = 100 u.t.

T_p(REG) = 100 u.t. // Tiempo de propagación de un registro.

T_p(REGFILE) = 200 u.t. // Tiempo de lectura del banco de registros

T_p(ALU-slow) = 800 u.t. // Tp de la ALU para las operaciones/funciones lentas: `ADD`, `SUB`, `CMP*`.

T_p(ALU-quick) = 400 u.t. // Tp de la ALU para las operaciones/funciones rápidas: cualquier otra distinta de `ADD`, `SUB`, `CMP*`.

T_{acc}(MEMORY) = 1000 u.t. // Tiempo de acceso (para la lectura o escritura) a la memoria

T_p(AND-2) = T_p(OR-2) = 20 u.t.

T_p(NOT) = 10 u.t.

El tiempo de propagación de un bloque combinacional (T_p) y el tiempo de acceso a memoria para realizar una lectura (T_{acc}) es el tiempo desde que están estables todas las entradas necesarias hasta que se estabilizan las salidas requeridas al valor correcto para las entradas aplicadas. Desconocemos como se han implementado internamente los bloques (y podría ser de forma diferente a los vistos en clase). Recordad que un registro con señal de carga (Ld), REGwLd, está construido con un REG y un MUX-2-1 (no os damos el esquema interno del REGwLd, porque lo tenéis que saber).

a) T_c correspondiente al nodo de **D** (decode).

T_c=

b) T_c correspondiente al nodo de **Addi**.

T_c=

c) T_c correspondiente al nodo de **Ldb**.

T_c=

Ejercicio 5 *(0.9 puntos)*

Indicad qué cambios se producen en el estado del computador SISC Von Neumann después de ejecutar cada una de las instrucciones de la tabla suponiendo que antes de ejecutarse cada una de ellas el PC vale `0x26AC`, el contenido de todos los registros pares es `0x6789` y el de los registros impares es `0x0123`, el byte contenido en todas las direcciones pares de la memoria es `0x64` y el de todas las impares es `0x32` y el contenido de todos los puertos de entrada es `1` y el de los de salida es `2`. Utilizad la notación MEM_b[0x…]=0x... y/o MEM_w[0x…]=0x... para indicar cualquier cambio en la memoria y PORTIN[0x...]=0x... para los puertos de entrada y PORTOUT[0x...]=0x... para los de salida.

Instrucción a ejecutar	Cambios en el estado del computador
`STB -9(R2), R4`	
`JALR R0, R5`	
`LD R6,13(R1)`	

Apellidos: ..... Nombre: ..... Grup: .....

Ejercicio 6 *(3.4 puntos)*

Debido a que es muy frecuente en programas tener que realizar una copia completa de un vector de una posición de memoria a otra, se ha decidido crear una nueva instrucción que haga esto.

Completad el diseño del SISC Von Neumann para que pueda ejecutar, además de las 25 instrucciones originales SISA, una nueva instrucción **COPYVECW** que permita copiar un vector de words de cualquier longitud de una posición de memoria a otra. Y que tiene el formato y codificación, la sintaxis ensamblador y la semántica siguientes:

Codificación: `1011 aaa bbb ccc xxx`
Sintaxis: `COPYVECW Ra, Rb, Rc`
Semántica: `PC=PC+2; MEMw[Rb]=MEMw[Ra]; Ra=Ra+2; Rb=Rb+2; Rc=Rc-1; if (Rc!=0) {PC=PC-2};`

Por ejemplo, la instrucción **COPYVECW** se usa para copiar los elementos de tipo word de un vector origen, cuya dirección inicial está en el registro `Ra` a un vector destino cuya dirección inicial está en el registro `Rb`. El registro `Rc` contiene la longitud del vector (con Rc_u>0). Supongamos que queremos copiar un vector de 25 elementos que está almacenado en la posición de memoria `0x1520` a la posición de memoria `0x6500`. En los siguientes códigos podéis observar cómo se escribiría el código en el SISA actual y como con la nueva instrucción.

Código original

```
        MOVI  R0, lo(0x1520)
        MOVHI R0, hi(0x1520)
        MOVI  R1, lo(0x6500)
        MOVHI R1, hi(0x6500)
        MOVI  R2, 25
bucle:  LD    R7, 0(R0)
        ST    0(R0), R7
        ADDI  R0, R0, 2
        ADDI  R1, R1, 2
        ADDI  R2, R2, -1
        BNZ   R2, bucle
```

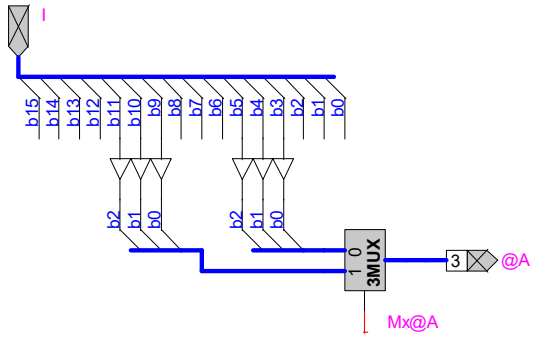
Código con la nueva instrucción

```
MOVI     R0, lo(0x1520)
MOVHI    R0, hi(0x1520)
MOVI     R1, lo(0x6500)
MOVHI    R1, hi(0x6500)
MOVI     R2, 25
COPYVECW R0, R1, R2
```

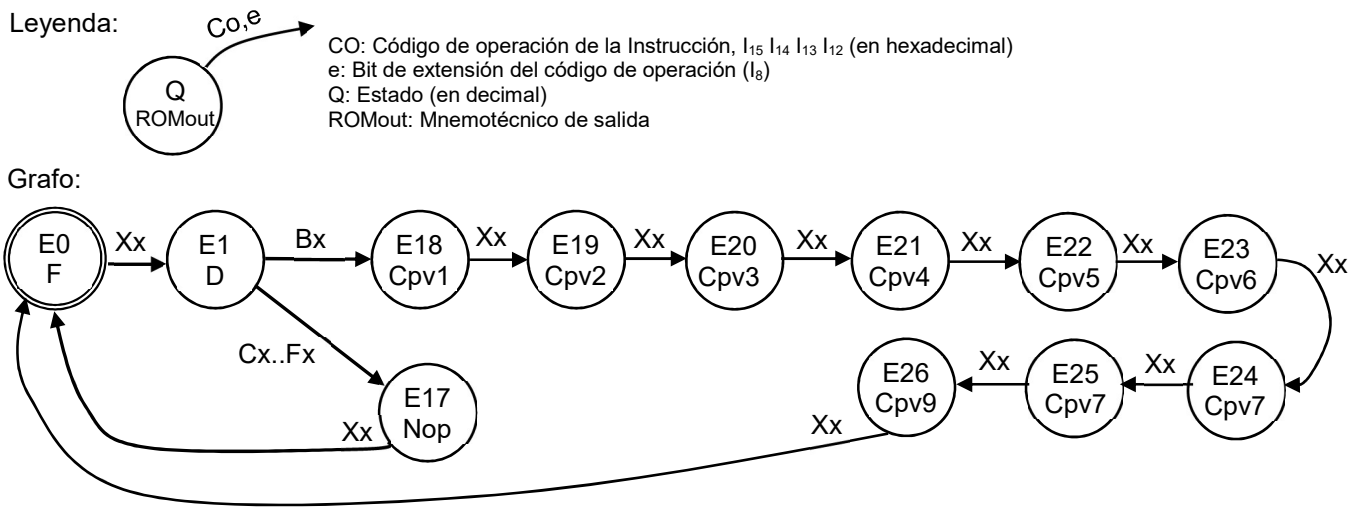
Aunque técnicamente no es necesario modificar el hardware de la UPG ni la UCG para añadir esta nueva instrucción, para simplificar la resolución del problema y que se ejecute en menos ciclos hemos modificado la UCG y la UPG con los siguientes cambios. Se ha añadido una funcionalidad nueva a la ALU que resta el valor que recibe por la entrada X en una unidad (W=X-1). Esta funcionalidad se ha añadido a una de les operaciones libres del bloque de Misc (miscelánea) de la ALU. Para usar esta nueva funcionalidad la señal OP debe valer 10 y F=100. La siguiente tabla muestra la lista de funcionalidades de la ALU con la operación añadida.

F			OP			
b_2	b_1	b_0	11	10	01	00
0	0	0	---	X	CMPLT (X,Y)	AND(X,Y)
0	0	1	---	Y	CMPLE (X,Y)	OR(X,Y)
0	1	0	---	MOVHI (X,Y)	---	XOR(X,Y)
0	1	1	---	X&(~1)	CMPEQ (X,Y)	NOT(X)
1	0	0	---	**X-1**	CMPLTU (X,Y)	ADD(X,Y)
1	0	1	---	---	CMPLEU (X,Y)	SUB(X,Y)
1	1	0	---	---	---	SHA(X,Y)
1	1	1	---	---	---	SHL(X,Y)

También se ha modificado la UCG, añadiendo un multiplexor MUX-2-1con señal de selección *Mx@A* para generar la señal *@A* que permita escoger entre los bits `I<11..9>` (`Ra`) y los bits `I<5..3>` (`Rc`). La siguiente figura muestra la parte de la Unidad de Control que ha cambiado respecto de la original.



Para añadir la nueva instrucción **COPYVECW** se han usado 9 nodos nuevos. El siguiente fragmento del grafo de estados de la figura correspondiente al circuito secuencial de la unidad de control corresponde a los estados de esta nueva instrucción.



Se pide:

a) Completad el contenido de la siguiente tabla que indica, mediante una fila para cada nodo del grafo de estados de la unidad de control, la acción (o acciones en paralelo) que se realiza en el computador en cada uno de los 11 ciclos/nodos que requiere la ejecución de la nueva instrucción **COPYVECW** (Fetch, Decode, y los 9 ciclos/nodos de la ejecución propiamente dicha): F, D, Cpv1, Cpv2, …, Cpv9. Para especificar las acciones usad el mismo lenguaje de transferencia de registros que en la documentación. El contenido de las filas/nodos de la tabla a rellenar deberá ser coherente con el apartado b). *(1.4 puntos)*

Nodo	Mnemotécnico	Acciones
E0	F	IR ← Mem_w[PC] // PC ← PC+2
E1	D	RX ← Ra // RY ← Rb // R@ ← PC+SE(N8)*2
E18	Cpv1	R@ ← [ ]
E19	Cpv2	Rb ← [ ] // RY ← Rb
E20	Cpv3	R@ ← RY // RY ← Rb // Rb ← RY
E21	Cpv4	[ ] ← RY // RY ← Rb
E22	Cpv5	RX ← Ra // Ra ← RY
E23	Cpv6	RX ← Ra // Ra ← [ ]
E24	Cpv7	Rb ← [ ]
E25	Cpv8	R@ ← [ ] // [ ]
E26	Cpv9	Rc ← [ ] // if ([ ]) [ ]

b) Completad (poniendo 0, 1 o x en cada bit) las celdas de la tabla que especifican el contenido de la ROM_OUT para las direcciones de los nodos que se han usado para implementar la nueva instrucción **COPYVECW** (del 18 a la 26). **Poned x siempre que el valor de un bit no importe**. Los valores de la tabla indicados son correctos y os pueden servir de ayuda para contestar el apartado anterior *(1 punto)*

Apellidos: ..... Nombre: ..... Grup: .....

@ROM	Mx@A	Bnz	Bz	WrMem	RdIn	WrOut	WrD	LdIr	Byte	R@/Pc	Alu/R@	Pc/Rx	Ry/N	P/I/L/A_1 P/I/L/A_0	OP_1 OP_0	MxN_1 MxN_0	MxF	F_2 F_1 F_0	Mx@D_1 Mx@D_0	***Nodo***
18		0	0	0	0	0	0	0	x	x	x			x x	1 0	x x	1			Cpv1
19		0	0	0	0	0	1	0	0	1	x			0 1	x x	x x	x			Cpv2
20		0	0	0	0	0	1	0	x	x	x			0 0	1 0	x x	1			Cpv3
21		0	0	1	0	0	0	0	0	1	x			x x	x x	x x	x			Cpv4
22		0	0	0	0	0	1	0	x	x	x			0 0	1 0	x x	1			Cpv5
23		0	0	0	0	0	1	0	x	x	x			0 0	0 0	1 1	1			Cpv6
24		0	0	0	0	0	1	0	x	x	x			0 0	0 0	1 1	1			Cpv7
25		0	0	0	0	0	0	0	x	x	x			x x	0 0	1 1	1			Cpv8
26		1	0	0	0	0	1	x	x	x	0			0 0	1 0	x x	1			Cpv9

c) Completad la dirección o contenido, según corresponda, de la ROM_Q+ del SISC Von Neumann. (0.2 puntos)

`ROM_Q+[0x0A8]=` `0x` `ROM_Q+[` `0x` `]=0x0F`

d) Sabiendo que la ejecución de la nueva instrucción **`COPYVECW`** se realiza en 11 ciclos (F, D y 9 nodos nuevos). Indica cuantas instrucciones se ejecutarían y cuantos ciclos se tardaría en ejecutar el programa de ejemplo del inicio del enunciado si ejecutásemos el código original y si ejecutásemos el programa con la nueva instrucción. *(0.8 puntos)*

Número instrucciones código original: Número instrucciones con la nueva instrucción:

Ciclos de ejecución código original: Ciclos de ejecución con la nueva instrucción: